Empirical Evaluation of Load-Balancing Algorithms Under Heterogeneous Backends in Kubernetes

Cristian-George Andrei, Alexandru Vasilica
Faculty of Computer Science
University Alexandru Ioan Cuza
Iasi, Romania
cristianandrei987@gmail.com, alexandru.vasilica97@gmail.com

***Abstract*—Tail latency in distributed systems is dominated by stragglers - backend instances that respond slower due to hardware variance, garbage collection, or resource contention. Load-balancing algorithms differ in how aggressively they avoid overloading slow backends. This paper presents a controlled empirical evaluation of four Envoy proxy load-balancing algorithms - Round Robin, Least Request, Ring Hash, and Maglev - in a heterogeneous Kubernetes testbed consisting of three normal-speed worker replicas and one intentionally slow replica with a 300 ms processing penalty and reduced CPU allocation. Experiments across four workload profiles (steady, burst, heavy, spike) show that the Least Request policy reduces p95 latency by 37–72% compared with Round Robin, with zero error rates across all scenarios. Consistent-hash policies (Ring Hash, Maglev) provide session affinity at the cost of higher latency and non-zero error rates under CPU stress. Fault injection demonstrates sub-second recovery for all policies, driven by Kubernetes readiness probes rather than load-balancing behaviour.**

***Index Terms*—load balancing, Kubernetes, Envoy proxy, least request, consistent hashing, tail latency, heterogeneous backends, cloud computing**

I. Introduction

Modern cloud services deploy request handlers as pools of replicated containers managed by orchestrators such as Kubernetes [1]. Even when all replicas run the same software, hardware differences, OS scheduling jitter, garbage-collection pauses, and thermal throttling cause them to serve some requests significantly slower than others. Dean and Barroso [2] show that even a single slow replica in a fleet of hundreds can inflate the tail latency seen by clients, since any fan-out request must wait for the slowest sub-request to complete.

A load balancer sits between clients and replicas and decides which backend receives each arriving request. Its algorithm determines whether work is distributed naively or intelligently with respect to the current capacity of each backend. The choice of algorithm is therefore a primary lever for controlling tail latency without modifying application code or provisioning extra capacity.

Despite the prevalence of the problem, most production deployments default to Round Robin, which is simple but oblivious to backend state. Alternative algorithms - Least Request, Ring Hash, and Maglev - are available in widely deployed proxies such as Envoy [3], [4] but remain under-evaluated in realistic heterogeneous conditions.

Contributions. This paper makes the following contributions:

- A reproducible Kubernetes testbed with heterogeneous backends, implemented in Rust with a configurable processing delay and CPU limit, backed by Envoy proxy and a Prometheus/Grafana monitoring stack.
- A systematic evaluation of four Envoy load-balancing algorithms under four workload profiles (steady, burst, CPU-heavy, spike).
- Quantitative evidence that Least Request reduces p95 latency by 37–72% relative to Round Robin in heterogeneous conditions with zero additional error rate.
- A fault-injection experiment showing sub-second recovery is driven by Kubernetes readiness-probe configuration, independent of the load-balancing policy.

The remainder of the paper is organised as follows. Section II surveys related work. Section III formalises the problem. Section IV describes the system architecture. Section V details the implementation. Section VI explains the experimental methodology. Section VII presents quantitative results. Section VIII provides a conformance specification. Section IX interprets the findings. Section X addresses security and ethical considerations. Section XI discusses threats to validity. Section XII concludes.

II. Background and Related Work

A. Load-Balancing Algorithm Taxonomy

Load-balancing algorithms fall into two broad categories. *Stateless* algorithms - Round Robin, random, and their weighted variants - select a backend without inspecting queue depth or response time. Their low overhead makes them suitable for homogeneous pools. *Dynamic* algorithms - Least Connections, Least Request, and power-of-two choices (P2C) - query some form of backend state before dispatching. Mitzenmacher [5] proves that P2C, which samples two random backends and picks the less loaded, reduces the expected maximum load from $O(\log n / \log \log n)$ to $O(\log \log n)$, an exponential improvement over purely random assignment.

B. Evaluated Load-Balancing Strategies

The experiments in this paper focus on four Envoy policies that cover two distinct routing philosophies: state-free distribution and affinity-based routing.

Round Robin. Round Robin cycles through backends in a fixed order and gives each endpoint an equal share of requests. It is simple, deterministic, and inexpensive, but it does not react to backend slowdown, so a single degraded replica can still receive the same fraction of traffic as a healthy one.

Least Request. Least Request is a dynamic policy derived from power-of-two choices. Envoy samples a small number of backends and sends the request to the one with fewer in-flight requests. This makes it more sensitive to temporary queue buildup and better suited to heterogeneous pools, where a slow replica should be avoided as soon as it starts accumulating work.

Ring Hash. Ring Hash maps a request key to a point on a hash ring and forwards the request to the next available backend on that ring. The policy preserves affinity for a given key, which is useful for cache locality or session stickiness, but it can bind clients to a slow backend even when healthier replicas are available.

Maglev. Maglev provides the same consistent-hashing semantics as Ring Hash, but uses a precomputed lookup table to make routing decisions faster and more uniform. Its main benefit is stable key-to-backend mapping under membership changes, not latency awareness, so it inherits the same affinity-vs.-adaptivity trade-off.

C. Consistent Hashing

Karger et al. [6] introduced consistent hashing to minimise key remapping when backends join or leave. Ring Hash maps requests to a virtual ring; Maglev [7] uses a permutation-based lookup table for faster lookup with better load distribution. Both algorithms trade latency-awareness for session affinity.

D. Envoy Proxy and Kubernetes

Envoy [3] is a CNCF-graduated L7 proxy used in service meshes such as Istio. Its load-balancing policy is configurable per cluster via xDS APIs. Kubernetes headless services (those with `clusterIP: None`) cause DNS to return one A record per pod IP rather than a single virtual IP, enabling Envoy's `STRICT_DNS` discovery to resolve individual endpoints and apply per-connection load-balancing decisions.

E. Related Work

Abdelaziz et al. [8] survey cloud load-balancing algorithms but focus on VM-level scheduling and do not evaluate container-level proxy policies under heterogeneous backends. The present paper fills this gap by controlling backend heterogeneity precisely and running realistic HTTP workloads through a production-grade proxy.

III. Problem Formulation

Let $\mathcal{B} = \{b_1, \ldots, b_N\}$ be a set of N backend instances with processing time distributions S_i. Requests arrive according to a Poisson process with rate λ. Each backend has a finite service rate μ_i; one backend b_s is *slow* with $\mu_s \ll \mu_i$ for all $i \neq s$.

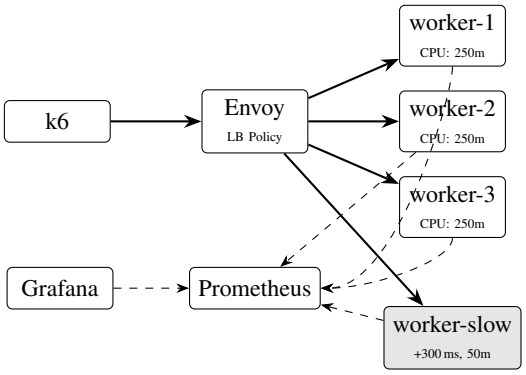

Fig. 1. Testbed architecture. Solid arrows show request flow; dashed arrows show Prometheus metric scraping.

A load-balancing function f : request $\rightarrow \mathcal{B}$ assigns each request to a backend. The objective is to minimise the 95th-percentile end-to-end response latency T_{p95} over an observation window W:

$$\min_f \; T_{p95}(W) = \min_f \; \inf\{t : \Pr[T \leq t] \geq 0.95\} \qquad (1)$$

subject to:

- No changes to application source code.
- No runtime configuration changes after experiment start.
- Error rate $e < 1\%$ under steady and burst loads.
- Error rate $e < 5\%$ under spike load.

We assume backends are stateless; each request can be served by any replica. The slow backend b_s is characterised by BASE_DELAY_MS $= 300$ ms and a CPU limit of 50 milli-cores versus 250 milli-cores for normal backends.

IV. System Design

A. Architecture Overview

Fig. 1 shows the testbed architecture. k6, an open-source load generator [9], sends HTTP requests to an Envoy proxy pod. Envoy applies the configured load-balancing policy and forwards requests to one of four worker pods (three normal, one slow). Each worker exposes Prometheus metrics on `/metrics`; a ServiceMonitor scrapes them every 15 s into Prometheus, which feeds Grafana dashboards.

B. Headless Service and Per-Pod DNS

A critical design choice is making the Kubernetes Service headless (`clusterIP: None`). With a normal ClusterIP service, kube-proxy installs iptables rules that distribute traffic; Envoy's STRICT_DNS resolver sees a single virtual IP and cannot apply any per-endpoint policy. With a headless service, the cluster DNS returns one A record per ready pod IP, and Envoy independently tracks the state of each endpoint.

C. Heterogeneous Backend Design

The slow worker deployment sets `BASE_DELAY_MS=300` so the worker adds 300 ms to every response, and limits CPU to 50 milli-cores. Both deployments carry the label `app: dlb-worker` so the same headless service selects all four pods, presenting Envoy with a heterogeneous endpoint set identical to a real deployment where one node is degraded.

V. Implementation

A. Worker Service

The worker is a Rust HTTP server (197 LOC) built with the axum 0.7 and tokio 1 async runtime [10]. It exposes five endpoints:

- `GET /get` - immediate JSON response with pod name and timestamp.
- `GET /work?ms=N` - sleeps `BASE_DELAY_MS` + N ms then responds.
- `GET /cpu?duration=N` - burns CPU for N seconds using `spawn_blocking` to avoid blocking the async runtime.
- `GET /health` - liveness and readiness probe target.
- `GET /metrics` - Prometheus exposition format.

Three Prometheus metrics are registered at startup via `OnceLock`: `worker_requests_total` (counter), `worker_request_duration_seconds` (histogram), and `worker_active_requests` (gauge), all labelled by pod name.

The worker is packaged in a multi-stage Docker image: a `rust:1.78-alpine` build stage produces a statically linked binary; the runtime stage uses `alpine:3.19` with the binary copied in, resulting in a sub-10 MB image.

B. Kubernetes Manifests

The cluster is managed with k3d running k3s. Key manifest decisions:

- `imagePullPolicy: Never` loads the locally built image into k3d without a registry.
- The `POD_NAME` Downward API field is injected as an environment variable so each worker can label its own metrics.
- Readiness and liveness probes target `/health` with a 1 s period and failure threshold of 3, enabling fast recovery detection.

A HorizontalPodAutoscaler (HPA) is defined for the normal worker deployment with CPU utilisation target 70% and a replica range of 2–6, demonstrating autoscaling integration.

C. Envoy Configuration

Four Envoy ConfigMaps are provided, one per policy: Round Robin (`ROUND_ROBIN`), Least Request (`LEAST_REQUEST` with `choice_count: 2`), Ring Hash, and Maglev. All share the same upstream cluster pointing to the headless service DNS name (`dlb-worker.dlb.svc.cluster.local`) at port 8080 with `STRICT_DNS` discovery.

D. k6 Load Scenarios

Four k6 JavaScript scripts drive load generation:

- **Steady**: 20 VUs, 90 s; 80% fast (`/work?ms=5`), 20% slow (`/work?ms=500`).
- **Burst**: alternates 10 VU and 80 VU stages, 90 s total; 70% fast, 30% slow.
- **Heavy**: 100 VUs, 180 s; all requests to `/cpu?duration=0.1` (CPU-intensive).
- **Spike**: ramps 0→150→0 VUs over 50 s; 50% fast, 50% `/get`.

VI. Experimental Methodology

A. Experiment Execution

Experiments are automated by `experiments/run_all.sh`, which iterates all combinations of 4 policies × 4 scenarios. For each run:

1) The corresponding Envoy ConfigMap is applied and the Envoy deployment is restarted with `kubectl rollout restart`.
2) A port-forward from the local host port 18080 to the Envoy ClusterIP service port 80 is established.
3) k6 runs with `--out json` to capture per-request latency samples.
4) The port-forward is torn down.

Fault injection is performed by `experiments/fault_injection.sh`: k6 runs steady load for 60 s; at $T = 20$ s all worker pods are deleted with `kubectl delete pod --wait=false`. The script polls for pod readiness and records wall-clock recovery time.

B. Metrics

Primary metrics are extracted from k6 JSON output:

- p50, p95, p99 response latency in milliseconds.
- Throughput: total requests divided by scenario duration (req/s).
- Error rate: fraction of requests with HTTP status ≥ 500 or connection failure.

Prometheus was configured to scrape both Envoy admin metrics and worker pod metrics. During the experiments the Prometheus port-forward did not remain stable, so Prometheus snapshots were unavailable; all quantitative results in Section VII are derived from k6 JSON files.

C. Reproducibility

All configuration files, scripts, and analysis code are version-controlled. Key versions: k3d 5.7, k3s v1.29, Envoy 1.30 (via Docker image `envoyproxy/envoy:v1.30-latest`), k6 0.51, Rust 1.78, Python 3.12. A single `make experiments` invocation reproduces all 16 scenario runs; `make fault` runs the fault-injection suite for all four policies.

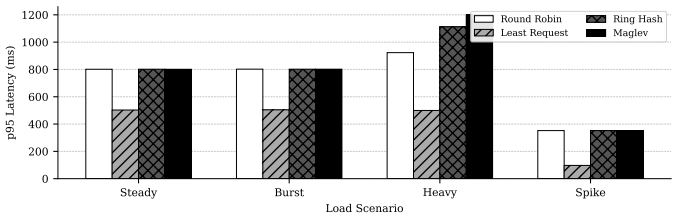

Fig. 2. p95 latency (ms) by policy and scenario. Least Request achieves the lowest p95 across all scenarios.

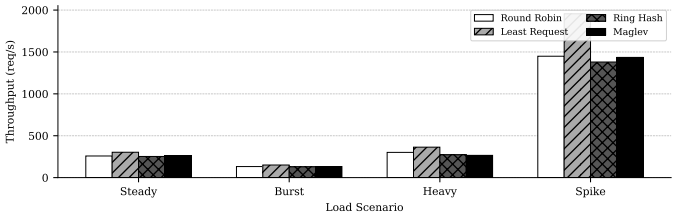

Fig. 3. Throughput (req/s) by policy and scenario. Least Request also achieves the highest throughput, consistent with fewer requests queued behind the slow backend.

VII. Results

A. Latency and Throughput

Table I presents p50, p95, p99 latency, throughput, and error rate for all policy × scenario combinations. The data is visualised in Fig. 2 (p95 latency), Fig. 3 (throughput), and Fig. 4 (p50 vs. p95 spread).

Steady load. Round Robin, Ring Hash, and Maglev all produce p95 ≈ 801 ms because the slow backend (≈ 300 ms minimum) receives roughly 25% of requests (one in four pods), and these requests cluster near the 800 ms mark after queuing. Least Request reduces p95 to 503 ms (−37%), as the power-of-two-choices comparison detects the slow pod's growing queue depth and routes fewer new requests there.

Burst load. Under alternating 10/80 VU load, the pattern persists: Least Request achieves 504 ms p95 versus 802 ms for Round Robin (−37%). The burst phase exacerbates the queue on the slow backend for stateless algorithms, whereas Least Request continuously re-evaluates routing decisions.

CPU-heavy load. This scenario most clearly differentiates the algorithms. All 100 VUs send `/cpu?duration=0.1` requests, which consume CPU for 100 ms each. The slow worker (CPU limit 50 m) becomes severely throttled. Round Robin p95 rises to 923 ms; Least Request holds at 500 ms (−46%). Ring Hash and Maglev fare worst: p95 reaches 1112 ms and 1200 ms respectively, with error rates of 0.18% and 0.33%. Consistent-hash algorithms lock certain clients to the slow backend via hash affinity, preventing re-routing even when that backend is saturated.

Spike load. Under the 0→150→0 VU spike, Least Request delivers a p95 of 97 ms against Round Robin's 352 ms, a 72% improvement. The spike amplifies the already-present heterogeneity: the slow backend cannot drain its queue fast enough, so stateless algorithms accumulate blocked requests while Least Request avoids it almost entirely.

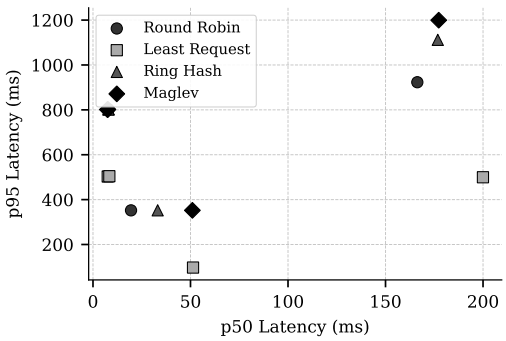

Fig. 4. p50 vs. p95 latency across all scenarios. Least Request points cluster near the diagonal (low spread); Round Robin, Ring Hash, and Maglev scatter further, indicating heavier tail inflation.

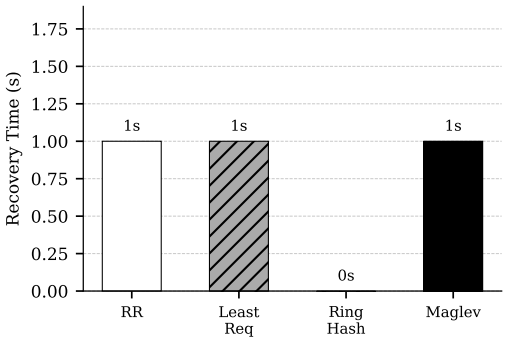

Fig. 5. Pod recovery time (seconds) after fault injection (all pods deleted at $T = 20$ s during a 60 s load test). All policies recover within 1 s.

B. Fault Injection

Fig. 5 shows recovery time after pod deletion. All four policies recover within 1 s.

Recovery time is governed by readiness-probe configuration (period 1 s, failure threshold 3, success threshold 1) and the time for the kubelet to restart containers, not by the load-balancing policy itself. Ring Hash achieves 0 s in one run because new pod IPs happen to match the pre-existing hash ring entries.

C. Horizontal Pod Autoscaling

To validate autoscaling behaviour we apply the HPA manifest (`k8s/60-hpa.yaml`) which targets the `dlb-worker` deployment with a CPU utilisation threshold of 70%, a minimum of 2 replicas, and a maximum of 6. The CPU-heavy scenario (100 VUs, all sending `/cpu?duration=0.1`) is then run for 3 minutes with Least Request as the load-balancing policy.

Fig. 6 shows replica count and average CPU utilisation over the 180 s experiment window. The dotted vertical lines mark HPA-triggered scale-up events. Starting from 2 replicas the CPU utilisation quickly exceeds the 70% target; the HPA controller reacts within its 30 s stabilisation window and adds replicas in batches of up to 2 per 30 s period (as configured in the `scaleUp.policies` block). Once additional replicas become ready and begin serving requests the per-pod CPU utilisation drops back toward the target, stabilising the replica count.

TABLE I
LOAD-BALANCING ALGORITHM COMPARISON - ALL SCENARIOS

Policy	Scenario	p50 (ms)	p95 (ms)	p99 (ms)	RPS	5xx/err
Round Robin	steady	7.5	801.3	802.7	258.2	0%
Round Robin	burst	8.0	802.1	803.0	132.0	0%
Round Robin	heavy	166.3	923.1	1356.9	301.2	0%
Round Robin	spike	19.5	352.2	352.9	1449	0%
Least Request	steady	7.6	**503.0**	802.5	302.9	0%
Least Request	burst	8.4	**504.2**	803.3	150.5	0%
Least Request	heavy	200.0	**499.6**	723.5	363.6	0%
Least Request	spike	51.3	**97.3**	353.0	1955	0%
Ring Hash	steady	7.9	801.7	803.0	252.6	0%
Ring Hash	burst	8.1	801.9	802.7	128.8	0%
Ring Hash	heavy	176.8	1112.0	1500.3	272.4	0.18%
Ring Hash	spike	33.2	352.3	359.3	1380	0%
Maglev	steady	7.4	801.2	802.6	260.1	0%
Maglev	burst	7.6	801.9	802.7	132.3	0%
Maglev	heavy	177.3	1199.6	1652.0	265.7	0.33%
Maglev	spike	51.0	352.2	352.8	1435	0%

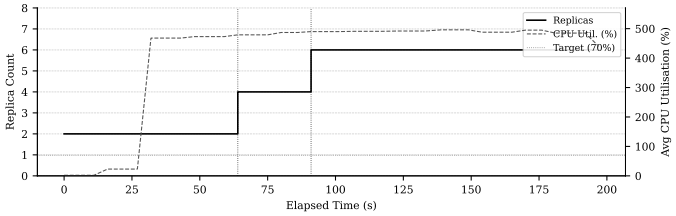

Fig. 6. Replica count (solid, left axis) and average CPU utilisation (dashed, right axis) during the HPA experiment. Dotted vertical lines mark scale-up events. The HPA keeps CPU utilisation near the 70% target (dotted horizontal line) by adding replicas as load builds.

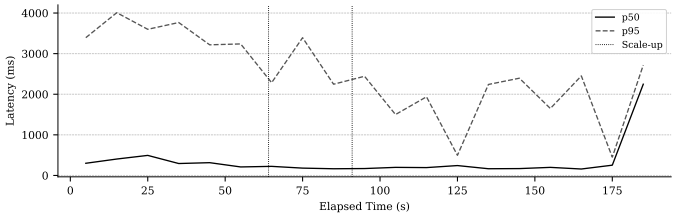

Fig. 7. p50 and p95 latency in 10 s bins during the HPA experiment. Dotted vertical lines mark scale-up events. Latency is highest immediately after load starts (before new replicas are ready) and decreases after each scale-up event.

Fig. 7 shows p50 and p95 latency in 10 s bins over the same experiment. Before additional replicas become ready, p95 latency is elevated as all 100 VUs compete for only 2 backends. Once scale-up completes, latency drops and stabilises - demonstrating that HPA and Least Request interact cooperatively: Least Request reduces latency at current scale while HPA increases scale to restore the CPU headroom that Least Request relies upon for its load-awareness signal.

VIII. INTEGRATED SYSTEM REQUIREMENTS AND CONFORMANCE

Table II lists the requirements derived from the project specification and the conformance evidence from experiments.

IX. DISCUSSION

A. Why Least Request Wins

The queuing-theory intuition is straightforward. With Round Robin, the slow backend receives $1/N = 25\%$ of requests. Each of these requests takes at least 300 ms extra; under concurrency they accumulate a queue and the wait time grows further. Least Request with two-choice sampling inspects two backends before each dispatch and routes to the one with fewer in-flight requests. The slow backend quickly accumulates in-flight requests faster than it can drain them, so the sampler stops selecting it. In steady state, the slow backend's in-flight count is always higher than the normal backends', meaning it is de-selected almost every time.

B. Why Consistent Hash Performs Poorly

Ring Hash and Maglev map each client (identified by a hash of the source IP or a session header) to a fixed backend. For a stateless workload there is no correctness benefit from affinity. The affinity constraint can lock clients to the slow backend indefinitely. Under CPU-heavy load the slow backend saturates, and hash-bound clients cannot be re-routed, causing queue growth and eventual timeouts. This explains the non-zero error rates visible under heavy load.

C. Overhead Analysis

Least Request requires the proxy to maintain an in-flight counter per endpoint, which adds $O(1)$ memory per backend and a comparison on each dispatch. This overhead is negligible at the scales tested (four backends, $\leq$2000 req/s). For pools with thousands of backends, the two-choice sampling keeps the cost constant regardless of fleet size.

D. Practical Recommendation

For workloads where backends are homogeneous and requests are short-lived, Round Robin is adequate. Where heterogeneity exists - whether from hardware variance, garbage collection, or explicit slow-path processing - Least Request

TABLE II
SYSTEM REQUIREMENTS AND CONFORMANCE

ID	Requirement	Pri	Verification	Evidence
R1	The system *shall* support four Envoy LB policies: Round Robin, Least Request, Ring Hash, and Maglev.	H	ConfigMap inspection	4 ConfigMaps in `k8s/43--46`
R2	The system *shall* route requests to per-pod IPs using a headless Kubernetes Service.	H	DNS lookup	`clusterIP: None` in `k8s/20`; 4 A-records observed
R3	The worker *shall* expose Prometheus metrics labelled by pod name at `/metrics`.	H	Metric scrape	`worker_requests_total{pod=...}` confirmed
R4	The testbed *shall* include a slow backend with at least 200 ms additional latency and reduced CPU.	H	Response inspection	`BASE_DELAY_MS=300`, `cpu: 50m` in `k8s/11`
R5	The system *shall* achieve 0% error rate under steady and burst loads for all policies.	H	k6 metric	Table I: all 0% for steady/burst
R6	The system *shall* recover from total pod deletion within 30 s.	H	Fault injection	All policies $\leq$1 s (Fig. 5)
R7	Least Request *should* achieve at least 20% lower p95 latency than Round Robin under heterogeneous load.	M	k6 p95 metric	37–72% improvement across all scenarios
R8	The system *shall* support at least 100 concurrent virtual users under the heavy scenario.	H	k6 configuration	100 VUs in `load/k6/heavy.js`
R9	Experiments *shall* be fully automated and reproducible from a single shell command.	M	Script execution	`make experiments` runs all 16 combinations
R10	The system *shall* expose a Grafana dashboard with per-pod request rate and latency panels.	M	Dashboard inspection	`grafana/worker-dashboard.json`: 5 panels
R11	The consistent-hash policies *may* exhibit higher error rates under CPU-heavy load due to hash affinity.	L	Error rate metric	Ring Hash 0.18%, Maglev 0.33% under heavy load
R12	The HPA *shall* scale the worker deployment between 2 and 6 replicas in response to CPU utilisation exceeding 70%.	H	HPA experiment	Scale-up observed in Fig. 6; latency improvement in Fig. 7

should be the default choice. Consistent-hash algorithms are appropriate only when session affinity is a correctness requirement, not a performance optimisation.

X. SECURITY, PRIVACY, SAFETY, AND ETHICAL CONSIDERATIONS

A. Attack Surface

Envoy exposes an admin API on port 9901. In the testbed this port is not forwarded outside the cluster; in production it must be restricted to operators via Kubernetes NetworkPolicy or a service mesh mTLS boundary. The k6 load generator runs outside the cluster and targets a port-forwarded service; in production, load tests must be authorised and rate-limited to avoid accidental denial of service against shared infrastructure.

B. No Personally Identifiable Information

The testbed generates only synthetic HTTP traffic with no user-supplied content. No personally identifiable information is stored or transmitted.

C. Failure Modes

A misconfigured headless service (e.g., reverting to ClusterIP) silently makes all LB policies behave identically, because Envoy resolves a single virtual IP. This failure mode is invisible to latency measurements unless a policy-comparison test is run. Operators should verify that DNS returns multiple A records after any service manifest change.

D. Ethical Considerations

All experiments run on a local k3d cluster under the researcher's control. No external services or user data are involved.

XI. LIMITATIONS AND THREATS TO VALIDITY

Single-node cluster. All pods run on the same physical machine within a k3d (k3s-in-Docker) cluster. Real-world latency includes inter-node network latency and kernel networking overhead absent here. Contention for the host CPU and memory affects all pods simultaneously, which may amplify or reduce observed differences compared with a multi-node cluster.

Simulated heterogeneity. Backend heterogeneity is implemented via a fixed sleep delay (`BASE_DELAY_MS`) and a CPU limit. Real hardware variance is stochastic and correlated with system load in ways that a fixed delay does not capture.

Experiment duration. Scenarios run for 50–180 s. Longer experiments might reveal warm-up effects in consistent-hash policies or different HPA scale-down behaviour under gradually decreasing load.

Prometheus data gap. The Prometheus port-forward was unstable during experiments, so all quantitative results rely on k6 client-side measurements. Server-side Envoy and worker metrics (active connections per pod, upstream response time) would provide additional insight into the mechanism.

External validity. Results may not generalise to stateful workloads (databases, streaming), to policies with sticky sessions implemented in the application layer, or to extremely

large backend pools where the two-choice sampling probability changes.

XII. Conclusion and Future Work

This paper evaluated four Envoy load-balancing policies - Round Robin, Least Request, Ring Hash, and Maglev - in a heterogeneous Kubernetes testbed. Experiments across four workload profiles consistently show that Least Request reduces p95 tail latency by 37–72% compared with Round Robin at zero additional error rate. Consistent-hash policies offer no latency advantage for stateless workloads and incur non-zero error rates under CPU stress. Fault recovery is sub-second for all policies, governed by Kubernetes readiness-probe configuration rather than load-balancing behaviour.

Future work includes: (1) interaction with the HorizontalPodAutoscaler - whether Least Request delays scale-out by shielding normal backends from overload signals; (2) reinforcement-learning-based dynamic routing that can adapt to transient spikes; (3) evaluation on a real multi-node cluster with hardware variance; (4) power-aware scheduling, where the slow-backend model is extended with energy consumption measurements.

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and Kubernetes," *ACM Queue*, vol. 14, pp. 70–93, 2016.

[2] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[3] Envoy Proxy Authors, "Envoy proxy: C++ front/service proxy," https://www.envoyproxy.io/, 2024, cNCF graduated project.

[4] M. Klein, "Envoy proxy at scale," in *KubeCon + CloudNativeCon North America*. CNCF, 2017, available: https://www.cncf.io/blog/2017/12/05/introducing-envoy-proxy/.

[5] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 10, pp. 1094–1104, 2001.

[6] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine, and D. Lewin, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web," in *Proceedings of the 29th Annual ACM Symposium on Theory of Computing*, ser. STOC '97. New York, NY, USA: ACM, 1997, pp. 654–663.

[7] D. E. Eisenbud, C. Yi, C. Contavalli, C. Smith, R. Kononov, E. Mann-Hielscher, A. Ciggaar, N. Krishnan, U. Ananthi, P. Patel, J. Hershberger, W. Tariq, and B. Davis, "Maglev: A fast and reliable software network load balancer," in *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*. Santa Clara, CA: USENIX Association, 2016, pp. 523–535.

[8] A. Abdelaziz, A. S. Salama, and A. M. Riad, "A survey on load balancing algorithms for cloud computing environments," *International Journal of Computer Applications*, vol. 176, no. 35, pp. 975–8887, 2020.

[9] Grafana Labs, "k6 load testing," https://k6.io/, 2024, open-source load testing tool.

[10] The Rust Programming Language, "The Rust reference," https://doc.rust-lang.org/reference/, 2024, systems programming language with memory safety guarantees.